

Priyanshu Sharma CU240250980

```
# write a program to read , display, and write an image using openCV in python.

import cv2
from google.colab.patches import cv2_imshow

# Read the image
image = cv2.imread("/content/images.jpg")

# Display the image
cv2_imshow(image)

# Write/Save the image
cv2.imwrite("/content/output.jpg", image)

print("Image saved successfully!")
```



Image saved successfully!

```
import cv2
from google.colab.patches import cv2_imshow

# Read the image
img = cv2.imread("/content/images.jpg")

# Display original image
print("Original Image:")
cv2_imshow(img)

# Resize the image
resized_img = cv2.resize(img, (500, 300))

# Display resized image
print("Resized Image:")
cv2_imshow(resized_img)

# Save the resized image
cv2.imwrite("/content/resized_image.jpg", resized_img)

print("Resized image saved successfully!")
```

Original Image:



Resized Image:



Resized image saved successfully!

```
# Write a program to convert a color image into grayscale and binary image.
import cv2
from google.colab.patches import cv2_imshow

# Read the color image
img = cv2.imread("/content/images.jpg")

# Display original color image
print("Original Color Image:")
cv2_imshow(img)

# Convert color image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display grayscale image
print("Grayscale Image:")
cv2_imshow(gray)

# Convert grayscale image to binary image
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Display binary image
print("Binary Image:")
cv2_imshow(binary)

# Save the images
cv2.imwrite("/content/grayscale.jpg", gray)
cv2.imwrite("/content/binary.jpg", binary)

print("Grayscale and binary images saved successfully!")
```

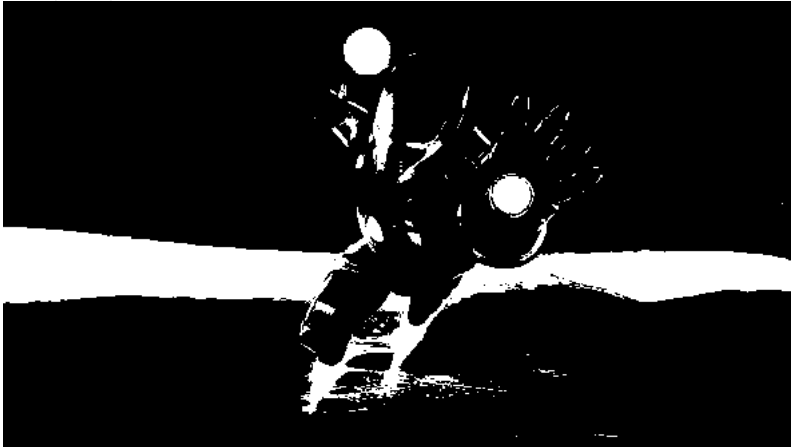
Original Color Image:



Grayscale Image:



Binary Image:



Grayscale and binary images saved successfully!

```
# 3. Write a program to compute and display the complement (negative) of an image.
import cv2
from google.colab.patches import cv2_imshow

# Read the image
img = cv2.imread("/content/images.jpg")

# Display original image
print("Original Image:")
cv2_imshow(img)

# Compute the complement (negative) of the image
negative = cv2.bitwise_not(img)

# Display negative image
print("Negative Image:")
cv2_imshow(negative)

# Save the negative image
cv2.imwrite("/content/negative.jpg", negative)

print("Negative image saved successfully!")
```

Original Image:



Negative Image:



Negative image saved successfully!

#4. Write a python program to Resize , Crop and rotate the Image.

```
import cv2
from google.colab.patches import cv2_imshow

# Read the image
img = cv2.imread("/content/images.jpg")

# Display original image
print("Original Image:")
cv2_imshow(img)

# Resize the image
resized = cv2.resize(img, (500, 300))

print("Resized Image:")
cv2_imshow(resized)

# Crop the image
# Format: image[y1:y2, x1:x2]
cropped = img[50:300, 250:400]

print("Cropped Image:")
cv2_imshow(cropped)

# Rotate the image by 90 degrees clockwise
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

print("Rotated Image:")
cv2_imshow(rotated)

# Save the images
cv2.imwrite("/content/resized.jpg", resized)
cv2.imwrite("/content/cropped.jpg", cropped)
cv2.imwrite("/content/rotated.jpg", rotated)

print("Images saved successfully!")
```


Original Image:

```
# 4.Split and merge the image.
import cv2
from google.colab.patches import cv2_imshow

# Read the image
img = cv2.imread("/content/images.jpg")

# Split the image into Blue, Green and Red channels
B, G, R = cv2.split(img)

# Display each channel
print("Blue Channel:")
cv2_imshow(B)

print("Green Channel:")
cv2_imshow(G)

print("Red Channel:")
cv2_imshow(R)

# Merge the channels back together
merged = cv2.merge([B, G, R])

# Display the merged image
print("Merged Image:")
cv2_imshow(merged)

# Save the merged image
cv2.imwrite("/content/merged.jpg", merged)

print("Image split and merged successfully!")
```



Cropped Image:



Rotated Image:



Images saved successfully!

Blue Channel:

